



Project Name – Reliance Digital Store Sales Analysis

Project Type - Exploratory Data Analysis (EDA)

Contribution - Individual

Name – Neeraj Singh Rawat

Tools used - Python, Pandas, Matplotlib, Seaborn

1. Data Understanding problem

This project focuses on performing Exploratory Data Analysis (EDA) on retail data from a Reliance Digital store. The goal is to analyze customer behavior, product performance, and sales trends using multiple related datasets.

The dataset used in this project consists of four interconnected tables: customers, orders, order items, and products. Together, these datasets provide a comprehensive view of the retail operations, including who the customers are, what products are being sold, and how transactions are carried out.

Import Data:

Firstly we have to import all our libraries which we are using in this project. Then import all dataset by using pandas.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
customers=pd.read_csv('messy_customers.csv')
order_item=pd.read_csv('messy_order_items.csv')
orders=pd.read_csv('messy_orders.csv')
products=pd.read_csv('messy_products.csv')
```

The project consist of 4 dataset:

a) Customers:

This dataset contain data of 520 customers with having unique id, name, city and their signup date.

customers

	customer_id	customer_name	city	signup_date
0	C0001	Customer_1	DELHI	2024-02-09
1	C0002	Customer_2	delhi	2025-04-20
2	C0003	Customer_3	Bangalore	2024-08-19
3	C0004	Customer_4	Pune	2023-08-08
4	C0005	NaN	Bengaluru	2024-03-04
...	...	...	...	...
515	C0391	Customer_391	Kolkata	2023-01-07
516	C0483	Customer_483	Delhi	2023-02-25
517	C0473	Customer_473	Bangalore	2023-12-05
518	C0417	Customer_417	Bangalore	2025-01-24
519	C0303	Customer_303	Kolkata	2024-03-23

520 rows × 4 columns

b) Order Item:

This dataset contain data of 2050 order item with having order item ID, order ID, product ID, quantity and their unit price.

order_item

	order_item_id	order_id	product_id	quantity	unit_price
0	1	O00380	P033	2.0	NaN
1	2	O00857	P010	1.0	NaN
2	3	O00589	P007	2.0	77415.0
3	4	O00112	P045	2.0	49066.0
4	5	O00322	P036	2.0	58380.0
...	...	...	...	...	...
2045	1166	O00667	P034	3.0	47907.0
2046	841	O00120	P050	4.0	41680.0
2047	399	O00053	P055	NaN	NaN
2048	545	O01182	P005	3.0	NaN
2049	1233	O00581	P047	1.0	89224.0

2050 rows × 5 columns

c) Orders:

This dataset contain data of 1240 orders with having order ID, customer ID, order date and their payment method.

orders				
	order_id	customer_id	order_date	payment_method
0	O00001	C0058	09/05/2024	Cash on Delivery
1	O00002	C0410	2024-02-01	upi
2	O00003	C0429	13/04/2025	Cash
3	O00004	C0272	2024-02-25	Debit Card
4	O00005	C0411	04-26-2025	credit card
...	...	...	...	...
1235	O01078	C0222	03-15-2025	Cash on Delivery
1236	O00944	C0014	04-08-2024	credit card
1237	O00128	C0383	03-31-2025	Credit Card
1238	O00966	C0053	23/02/2024	upi
1239	O00313	C0428	12-21-2024	Cash on Delivery

1240 rows × 4 columns

d) Products:

This dataset contains data of 90 products with having product ID, product name, category and there base price.

products				
	product_id	product_name	category	base_price
0	P001	Lenovo IdeaPad	Tablet	28151.0
1	P002	Samsung Tab	Smartphone	68660.0
2	P003	HP Pavilion	Accessories	NaN
3	P004	Lenovo Tab	tablet	NaN
4	P005	OnePlus	Tablet	NaN
...	...	...	...	...
85	P022	Samsung Tab	Accessory	NaN
86	P052	Xiaomi Redmi	smart phone	NaN
87	P010	iPhone 13	smart phone	NaN
88	P059	OnePlus	Accessories	NaN
89	P033	Wireless Mouse	laptop	47453.0

90 rows × 4 columns

2. Pre-processing data

Before performing the analysis, the datasets were cleaned and pre-processed to ensure accuracy, consistency, and reliability of the results. Removing duplicate values, check datatype of the column and convert it into proper datatype.

Drop Duplicates values form the datasets

```
customers.drop_duplicates(inplace=True)
```

```
order_item.drop_duplicates(inplace=True)
```

```
orders.drop_duplicates(inplace=True)
```

```
products.drop_duplicates(inplace=True)
```

Convert date into pandas datetime format

```
customers['signup_date'] = pd.to_datetime(customers['signup_date'])
```

Signup date column in customers table now in proper pandas date time format

```
orders['order_date'] = pd.to_datetime(orders['order_date'],format='mixed')
```

order_date in orders table are in mixed format so we use format function.

3. Handling Inconsistent Data

Inconsistent data can lead to incorrect analysis and misleading insights, so it is important to identify and resolve such issues. The datasets were examined for inconsistencies in formats, naming conventions, and values across all four tables (customers, orders, order items, and products).

One common issue is inconsistency in text data, such as different spellings, extra spaces, or mixed letter cases. These were standardized by converting text to a uniform format (such as lowercase or uppercase) and removing unnecessary spaces.

Same values may appear in slightly different forms like in our data (e.g. Bangalore, Bengaluru)

- a) In customer dataset city column has same values but in different forms like somewhere city written in small case (delhi), and somewhere written in uppercase(DELHI).

```
customers['city'] = customers['city'].str.title()
```

str is a module of pandas which operates string values and with the help title function first letter is capital and other are small.

Str is a module of pandas which operates string values and with the help of title function first letter is capital and other are small letter.

- b) Solve **inconsistent values** by using replace function.

```
: customers['city']=customers['city'].replace({'Bangalore':'Bengaluru'}).copy()
```

```
customers['city']
```

```
0      Delhi
1      Delhi
2  Bengaluru
3       Pune
4  Bengaluru
...
495     Delhi
496       Pune
497    Mumbai
498     Delhi
499    Kolkata
```

- c) **Handle inconsistent** values in product's category column convert text into proper format and replace spelling mistake values.

```
[104]: products['category']=products['category'].str.title()
products['category']=products['category'].replace({"Accessory":"Accessories","Smart Phone":"Smartphone"})
```

```
products['category']
```

```
0      Tablet
1  Smartphone
2  Accessories
3      Tablet
4      Tablet
...
75  Accessories
76  Smartphone
77  Accessories
```

- d) Handle Inconsistent values in orders table where payment method have inconsistent values.

```
: orders['payment_method']=orders['payment_method'].str.title()  
orders['payment_method']=orders['payment_method'].replace({"Cash On Delivery":"Cash"})
```

```
orders['payment_method']
```

```
0      Cash  
1      Upi  
2      Cash  
3  Debit Card  
4  Credit Card  
...  
1195     Upi  
1196     Upi  
1197     Upi  
1198     Upi  
1199     Upi
```

4. Handling Missing Values

Handling missing values is a crucial step in data pre-processing, as incomplete data can affect the accuracy and reliability of the analysis. Each dataset—customers, orders, order items, and products—was carefully examined to identify null or missing values.

- In cases where missing values were minimal and did not significantly affect the dataset, the corresponding rows were removed to maintain data integrity.
- For columns where missing values were important but limited in number, suitable imputation techniques were used. For example, numerical fields such as price or quantity were filled using measures like mean or median where appropriate.
- For categorical columns (such as product category or customer location), missing values were handled by either filling them with the most frequent value (mode) or labeling them as “Unknown” to retain the data without introducing bias.
- Fill randomly by using ffill (forward fill) or bfill (backward fill).

```
orders['payment_method']=orders['payment_method'].ffill().copy()
```

I fill payment method's null values by using forward fill function and we pass a copy of that column.

```
customers['city']=customers['city'].bfill().copy()
```

I fill city's null values by using backward fill function and we pass a copy of that column.

- Fill mean quantity in quantity column and mean unit price in unit price column.

```
order_item.fillna({'quantity':order_item['quantity'].mean(),'unit_price':order_item['unit_price'].mean()},inplace=True)
```

	order_item_id	order_id	product_id	quantity	unit_price
0	1	O00380	P033	2.000000	47453.000000
1	2	O00857	P010	1.000000	45038.245702
2	3	O00589	P007	2.000000	77415.000000
3	4	O00112	P045	2.000000	49066.000000

5. Merge Data

Since the project consists of multiple datasets—customers, orders, order items, and products—it was necessary to combine them into a single unified dataset to perform comprehensive analysis. Data merging allows us to view all relevant information about each transaction in one place, including customer details, product information, and order history.

The merging process was carried out using common keys present across the datasets:

- The customers dataset was linked with the orders dataset using customer_id, as each customer can place multiple orders.
- The order items dataset was merged with the products dataset using product_id to include product-related information such as name, category, and price.
- And finally merge both by order id by using inner join.

```
first = order_item.merge(products,on='product_id',how='outer')
```

```
second = orders.merge(customers,on='customer_id',how='outer')
```

```
df = first.merge(second,on='order_id',how='inner')
```

```
df
```

6. Transform Data

After merging the datasets, data transformation was performed to make the data more suitable for analysis and to extract meaningful features. This step helps convert raw data into a structured format that supports better insights and visualization.

Amount column was derived by multiplying price and quantity.

```
df['Amount'] = df['quantity'] * df['unit_price']
```

The **order date** column was further utilized to extract additional time-related features such as month, year.

```
df['year'] = df['order_date'].dt.year
```

```
df['month'] = df['order_date'].dt.month_name()
```

date	Amount	year	month
14	30940.0	2024	September
24	112604.0	2024	October
13	112604.0	2024	July
09	28151.0	2024	January
26	34936.0	2025	April
...	...	...	...
30	82478.0	2025	March
05	164956.0	2024	March
21	0.0	2024	October

7. Exploring Data & Visualization

>> Finding total revenue

```
#total Revenue  
total_revenue=df['Amount'].sum()  
print(f"₹{int(total_revenue)}")
```

₹165023180

>> Finding total quantity sold

```
#total quantity sold  
total_qty_sold = df['quantity'].sum()  
print(total_qty_sold)
```

3685

>> Monthly Revenue

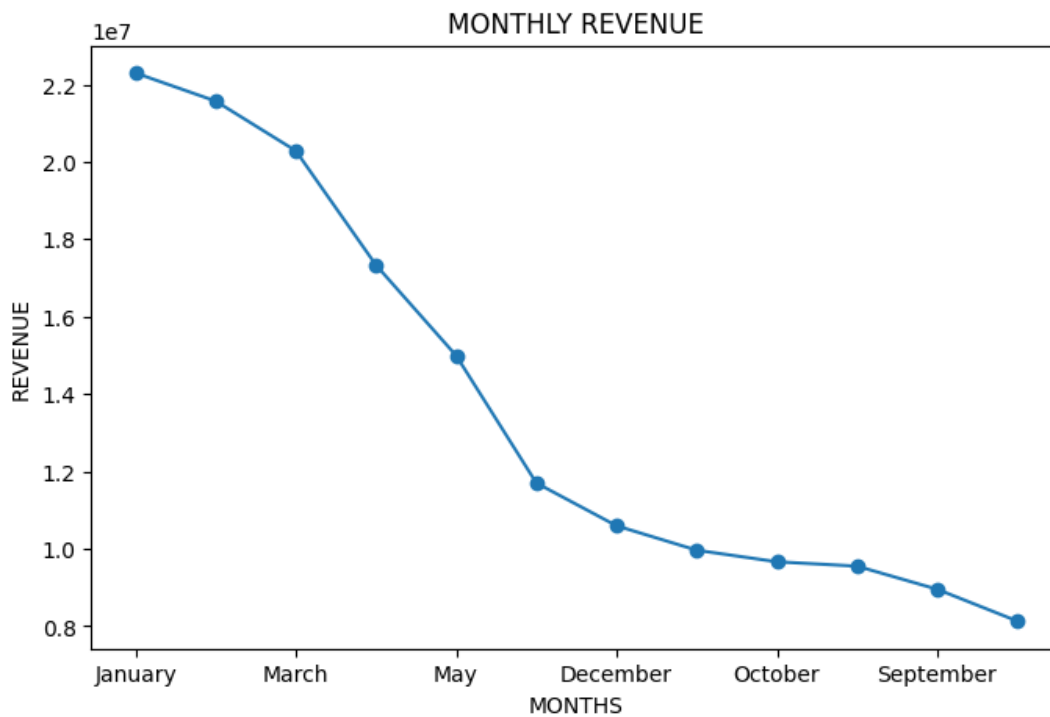
```
#revenue by month  
monthly_sales = df.groupby('month')['Amount'].sum().astype('int')  
print(monthly_sales.sort_values(ascending=False))
```

month	
January	22296015
April	21569575
March	20284421
February	17330222
May	14990969
June	11699719
December	10593763
July	9962422
October	9663569
November	9550624
September	8952460
August	8129417

It is clearly see that Quarter1 generates highest revenue from others

```
plt.figure(figsize=(8,5))  
monthly_sales.sort_values(ascending=False).plot(kind='line',marker='o')  
plt.title('MONTHLY REVENUE')  
plt.xlabel('MONTHS')  
plt.ylabel('REVENUE')
```

```
Text(0, 0.5, 'REVENUE')
```



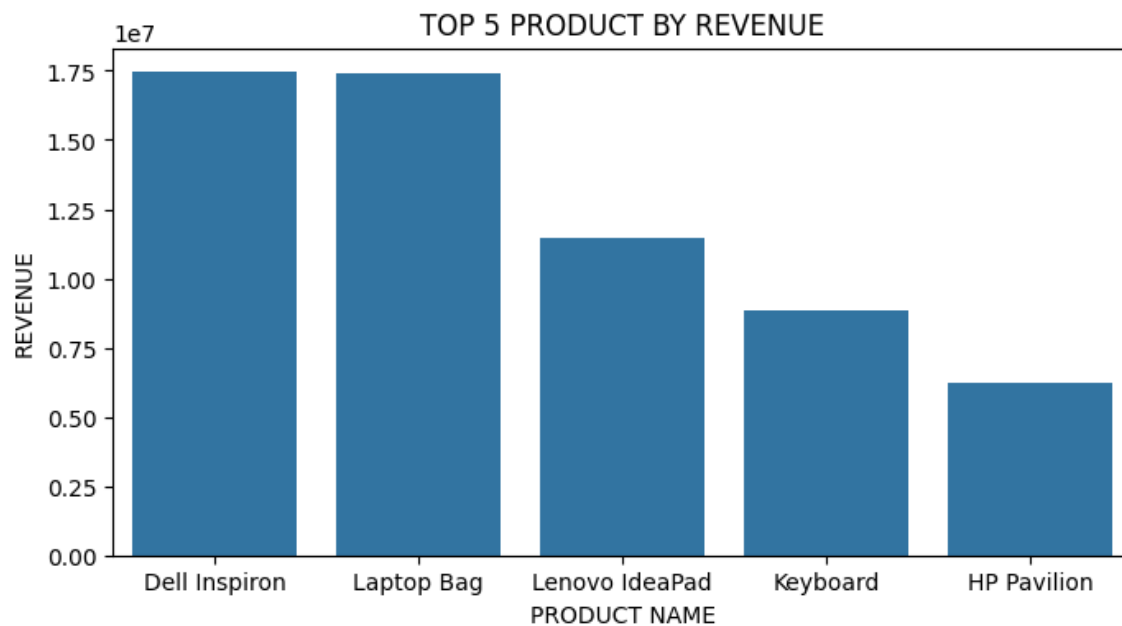
As shown in chart, we can say that January month generates the highest and August month has minimum revenue.

>> Top 5 Product by revenue

```
#top_product by revenue
top_product = df.groupby('product_name')['Amount'].sum().head(5).astype('int')
top_5_prod = pd.DataFrame(top_product.sort_values(ascending=False))
top_5_prod
```

Amount	
product_name	
Dell Inspiron	17443441
Laptop Bag	17389637
Lenovo IdeaPad	11479789
Keyboard	8818222
HP Pavilion	6206948

```
plt.figure(figsize=(8,4))
sns.barplot(x='product_name',y='Amount',data=top_5_prod)
plt.title('TOP 5 PRODUCT BY REVENUE')
plt.xlabel('PRODUCT NAME')
plt.ylabel('REVENUE')
```



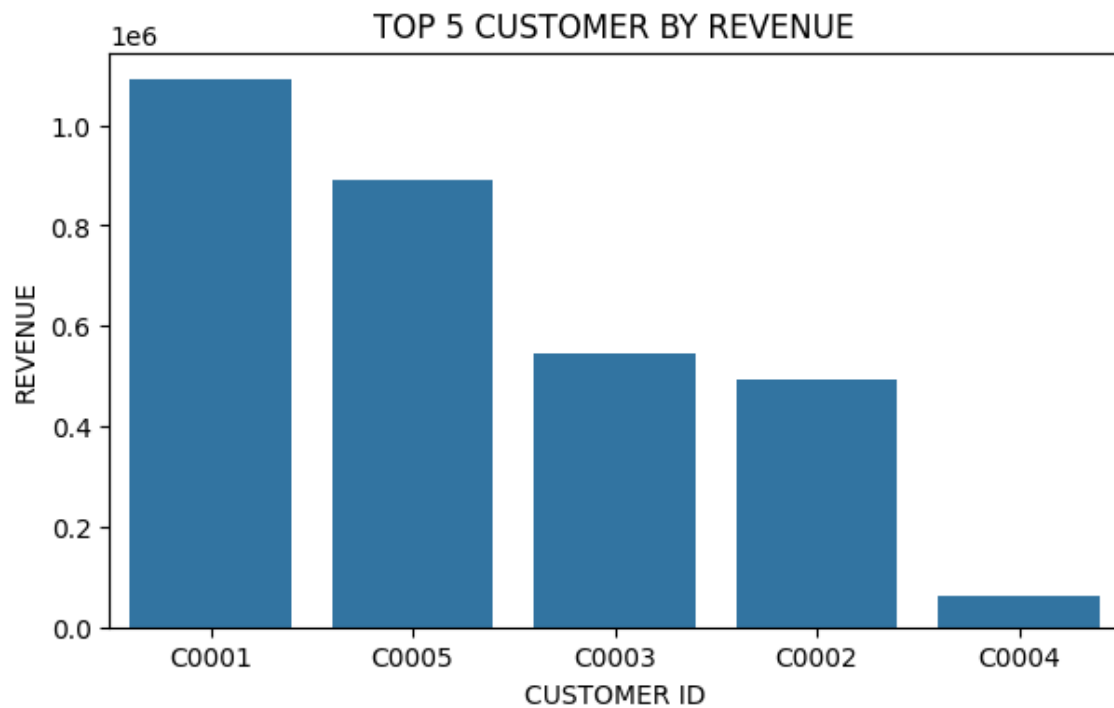
Dell Inspiron and Laptop Bag generates the highest revenue

>> Top 5 customer by revenue

```
top_customer = df.groupby('customer_id')['Amount'].sum().head(5).astype('int')
top_5_cust = pd.DataFrame(top_customer.sort_values(ascending=False))
top_5_cust
```

Amount	
customer_id	
C0001	1089958
C0005	890416
C0003	545229
C0002	494086
C0004	60996

```
plt.figure(figsize=(7,4))
sns.barplot(x='customer_id',y='Amount',data=top_5_cust)
plt.title('TOP 5 CUSTOMER BY REVENUE')
plt.xlabel('CUSTOMER ID')
plt.ylabel('REVENUE')
```



>> Yearly Revenue

```
#revenue by year
yearly_sales = df.groupby('year')['Amount'].sum().astype('int')
print(yearly_sales)
```

```
year
2024    120871668
2025     44151511
```

In 2024, sales are high as compare to the 2025.

>> Quantity Sold in City

```
citywise_qty = df.groupby('city')['quantity'].sum()
city_qty = pd.DataFrame(citywise_qty.sort_values(ascending=False))
city_qty
```

quantity	
city	
Delhi	1100
Mumbai	638
Bengaluru	550
Hyderabad	512
Pune	376
Kolkata	277
Chennai	232

Delhi is the city where sales are high.

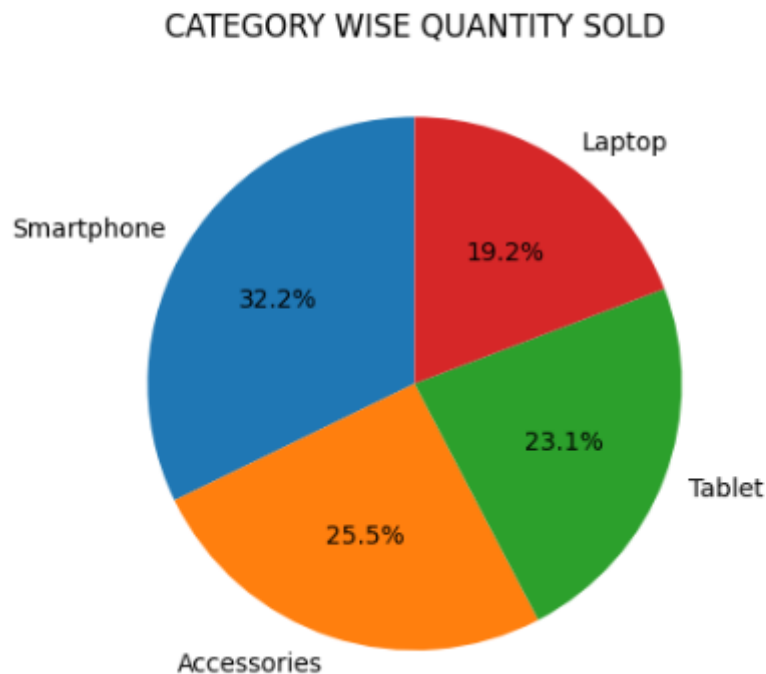
>> Quantity sold category wise

```
category_qty = df.groupby('category')['quantity'].sum()
category_wise_qty = pd.DataFrame(category_qty.sort_values(ascending=False))
category_wise_qty
```

quantity	
category	
Smartphone	1187
Accessories	938
Tablet	852
Laptop	708

```
plt.pie(
    category_wise_qty['quantity'],          # values
    labels=category_wise_qty.index,         # category names
    autopct='%1.1f%%',                     # percentage show karega
    startangle=90
)
plt.title('CATEGORY WISE QUANTITY SOLD')
```

Text(0.5, 1.0, 'CATEGORY WISE QUANTITY SOLD')



Smartphone category contribute the highest quantity sold

>> Revenue generated through each payment method

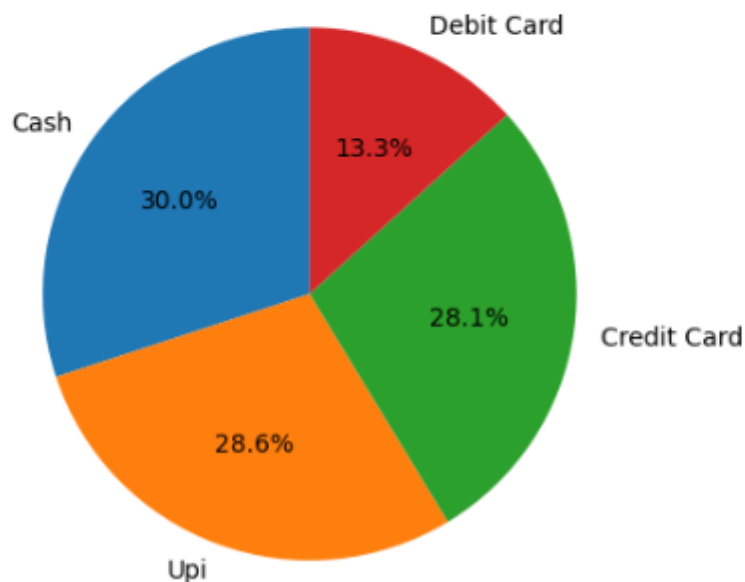
```
payment_methods = df.groupby('payment_method')['Amount'].sum().astype('int')
payment_wise_revenue = pd.DataFrame(payment_methods.sort_values(ascending=False))
payment_wise_revenue
```

	Amount
payment_method	
Cash	49578475
Upi	47240103
Credit Card	46314821
Debit Card	21889780

```
plt.pie(
    payment_wise_revenue['Amount'],          # values
    labels=payment_wise_revenue.index,      # category names
    autopct='%1.1f%%',                      # percentage show karega
    startangle=90
)
plt.title('PAYMENT METHOD DISTRIBUTION')
```

Text(0.5, 1.0, 'PAYMENT METHOD DISTRIBUTION')

PAYMENT METHOD DISTRIBUTION



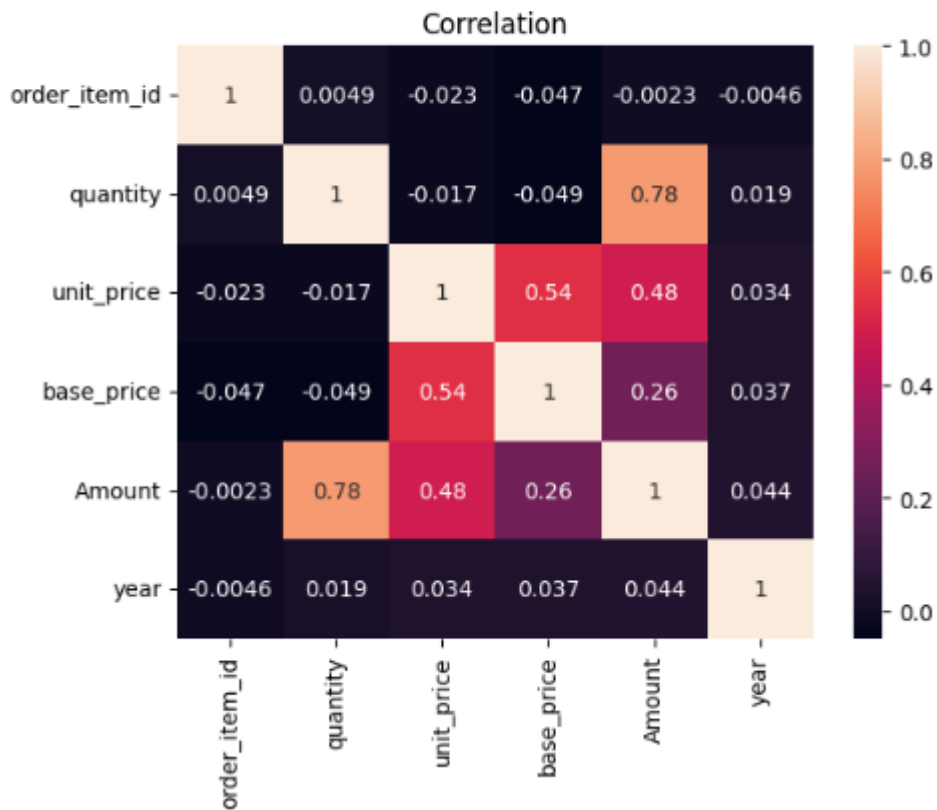
Most amount received by cash.

>>Correlation Heatmap

The heatmap shows the correlation between the numeric variables.

```
numeric_df = df.select_dtypes(include=np.number)
sns.heatmap(numeric_df.corr(),annot=True)
plt.title('Correlation')
```

```
Text(0.5, 1.0, 'Correlation')
```



>> There is a strong correlation (0.78) between quantity and amount. As quantity increase amount also increase.

>> There is a moderate correlation (0.54) between base price and unit price. Since it is positive, when the base price increases, the unit price generally tends to increases too.

8. Conclusion

- Dell Inspiron and laptop bags generated the highest revenue among all products, making them the top-performing products in terms of revenue contribution.
- Customers C0001 and C0005 are identified as the company's key customers because of their significant contribution to overall sales and revenue.
- Smartphones are the most sold product category, indicating a high customer demand and strong market preference for mobile devices.
- Delhi recorded the highest sales compared to all other cities, showing that it is the company's strongest market location.
- From a product perspective, Dell Inspiron is the most important product, while smartphones hold the highest importance among all product categories.
- The company achieved its highest sales performance during the first four months of the year, indicating strong business activity in the initial phase of the year.